Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра штучного інтелекту

Дисципліна «Аналітика великих даних»

Практичне заняття №4

«Зберігання та аналіз великих даних у NoSQL-сховищі MongoDB»

Виконав:
ст. гр. ДМм-26-1
Зелінський В.С.

Прийняв:
Золотухін О.В.

Харків – 2026

Зберігання та аналіз великих даних у NoSQL-сховищі MongoDB

Мета роботи

Ознайомитися з принципами організації NoSQL-сховищ та документною моделлю MongoDB; отримати практичні навички створення бази даних і колекцій, імпорту великого набору даних, виконання запитів, індексації та агрегування на прикладі 7 816 537 хвилинних свічок BTC/USD (Bitcoin Historical Data, 2012–2026 роки); навчитися використовувати MongoDB як сховище результатів обробки великих даних, отриманих у ПЗ2 (Spark) та ПЗ3 (виявлення аномалій).

Короткі теоретичні відомості

MongoDB – документна NoSQL-СКБД (поряд із key-value, колонковими і графовими системами): сервер `mongod` містить бази, база – колекції, колекція – документи у форматі BSON. Схема не нав'язується, тому в одній базі зберігаються свічки, ознаки Spark і аномалії різної структури. Для цієї роботи суттєві типи BSON Date (час UTC у мілісекундах) та Int64, відсутні в JSON: на Date спираються діапазонні фільтри за часом.

Запит `find(filter, projection)` фільтрує операторами `$gt`, `$lt`, `$in`, `$and`; Aggregation Framework пропускає документи через конвеєр стадій (`$match`, `$group`, `$sort`, `$project`, `$bucket`, `$lookup`).

Індекс – B-дерево ключів: без придатного індексу планувальник читає всю колекцію (COLLSCAN), з ним – лише потрібні ключі й документи (IXSCAN і FETCH). Ефект індексу вимірюється лічильниками `totalDocsExamined` і `executionTimeMillis` з `explain("executionStats")`. Для складених індексів діє правило ESR:

поля рівності, потім сортування, потім діапазону. Time-series колекція групує виміри у внутрішні бакети зі стовпцевим стисненням.

Результати виконання практичного заняття

1 Підготовка MongoDB

Через блокування `fastdl.mongodb.org` проксі сервер встановлено з conda-forge (micromamba), mongosh – з npm, PyMongo – через pip: MongoDB Community Server 8.3.7, mongosh 2.14.1, PyMongo 4.19.0, Python 3.13.18, pandas 3.0.5. Машина: Intel Xeon 2,10 ГГц (4 vCPU), ОЗП 15,7 ГіБ, Linux (glibc 2.39). Compass потребує дисплея, тому його операції виконано в mongosh. PyMongo підключається з `tz_aware=True`, щоб дати поверталися в UTC (файл `pz4_common.py`). Паралельно працювали задачі Spark (loadavg 5–12 при 4 vCPU), тому порівнюються лише варіанти одного прогону з прогрівом і медіаною.

2 Створення бази даних і колекцій

База `bigdata_db` і звичайні колекції створюються неявно першим `insert_many`, а time-series колекцію створено явно методом `create_collection` з параметром `timeseries` (поле часу `ts`, метадані `meta`, гранулярність `minutes`; файл `pz4_load.py`). Сім колекцій і їх розміри за `collStats` наведено в таблиці 1.

Таблиця 1 – Колекції бази bigdata_db (розміри в МіБ, 1 МіБ = 2^{20} Б)

Колекція	Призначення	Документів	size	storageSize	Індекси
candles_1m	хвилинні свічки, плоска схема (основна)	7 816 537	894,53	383,05	280,07
candles_1m_ts	ті самі свічки у time-series колекції	7 816 537	76,94	69,31	0,45
candles_1h	годинні свічки, плоска схема	130 276	13,17	6,56	2,72
candles_1d	добові свічки	5 429	0,75	0,30	0,25
candles_1d_buckets	доба з масивом 24 годинних свічок	5 429	11,68	5,25	0,14
spark_daily_features	добові ознаки, обчислені Spark у П32	5 429	2,04	0,96	0,14
anomalies	аномальні хвилини П33 (правило v2)	5 437	1,59	0,63	0,25

Розміри знято після створення індексів розділу 6 (для `candles_1m`: `_id_` 82,23, `ts_1` 90,43, `volume_-1` 107,41 МіБ; у time-series колекції лише автоматичний `meta_1_ts_1`). Кількість документів за `countDocuments` збігається з кількістю рядків джерел.

2.1 Обґрунтування схеми

Порівняно три схеми: плоский документ на хвилину (`candles_1m`), time-series колекцію (`candles_1m_ts`) і шаблон bucket (`candles_1d_buckets`, доба з масивом 24 годинних свічок); обсяг зберігання показано на рисунку 1. BSON плоскої колекції займає 894,53 МіБ (937,98 МБ, 120 Б на документ) – у 2,36 раза більше за вихідний CSV (396,85 МБ), бо кожен документ повторює імена полів; після стиснення на диску – 383,05 МіБ. Time-series колекція (7 817 бакетів зі стовпцевим стисненням) займає 69,31 МіБ, у 5,5 раза менше, а її індекс – 0,45 проти 90,43 МіБ для `ts_1`. Bucket-схема годинних свічок дає менший виграш (5,25 проти 6,56 МіБ), але зменшує кількість документів у 24 рази.

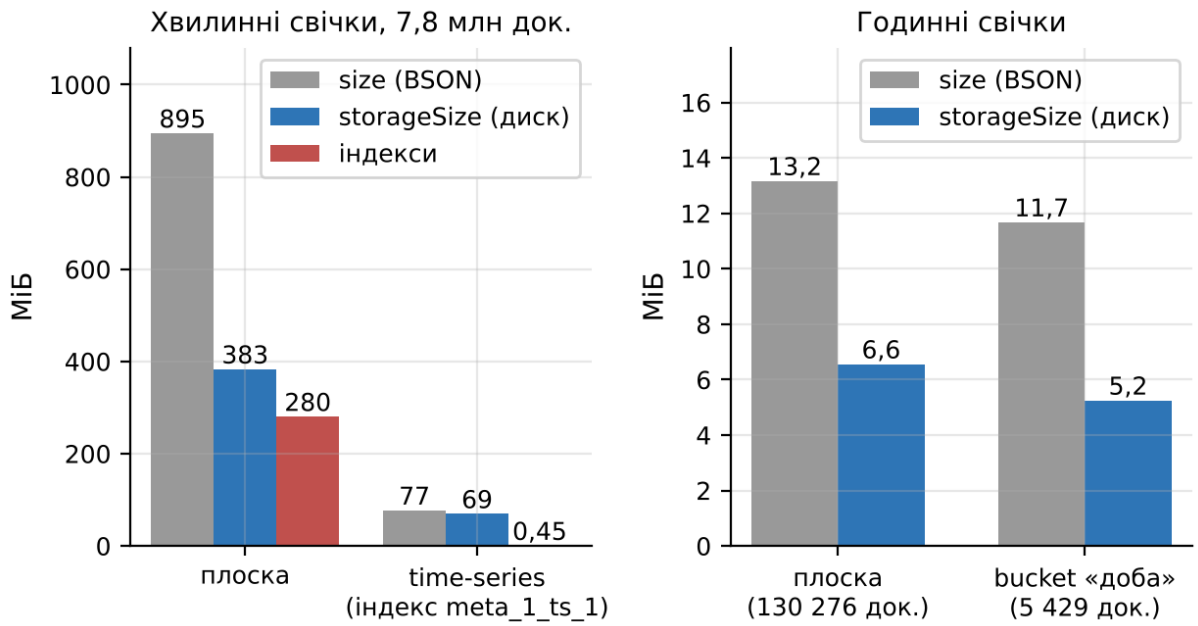


Рисунок 1 – Обсяг зберігання, МіБ: звичайна і time-series колекції хвилинних свічок, плоска і bucket-схема годинних свічок

Час трьох типових запитів до обох колекцій (прогрів, медіана з 5 запусків) показано на рисунку 2.

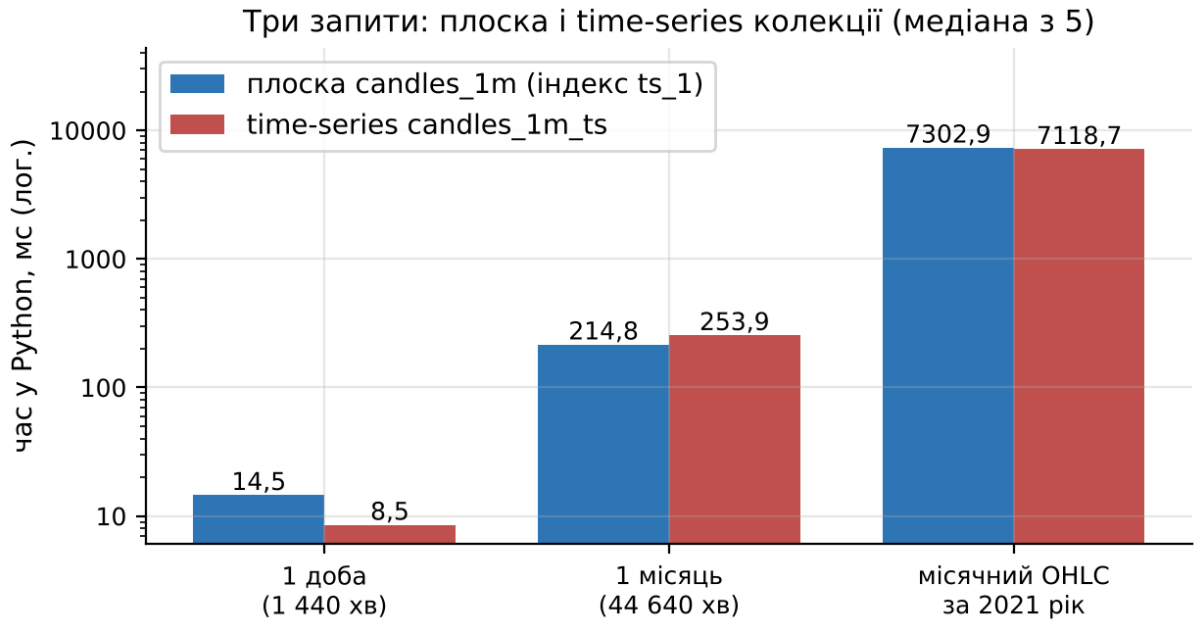


Рисунок 2 – Час запитів до плоскої і time-series колекцій (повний час у Python, медіана з 5 запусків)

Доба читається з time-series колекції швидше (8,5 проти 14,5 мс), місяць – повільніше (253,9 проти 214,8 мс), місячний OHLC за 2021 рік – однаково (7 118,7 і 7 302,9 мс); розкид запусків перевищує різницю медіан.

Тому основною обрано плоску колекцію: вона дозволяє вторинний індекс на будь-якому полі (`volume_-1`) і оновлення окремих хвилин, а `explain` рахує хвилини, а не бакети. Time-series колекція придатна для архіву.

3 Імпорт даних

3.1 Завантаження

Джерело – оброблений `btc_1min.parquet` (час UTC) і похідні годинний та добовий набори. Документи формуються з DataFrame і вставляються `insert_many(ordered=False)` пакетами по 20 000; `mongoimport` не використано, бо для CSV типи Date та Int64 довелося б описувати вручну.

Завантаження 7 816 537 документів у `candles_1m` тривало 117,7 с: 18,2 с – побудова документів у Python, 99,6 с – вставка (78,5 тис. док/с, 391 звернення по близько 2,4 МБ, що менше за межу повідомлення 48 МБ; інші розміри пакета не вимірювалися). Time-series колекція завантажена за 99,2 с (вставка 78,5 с, 99 638 док/с, на 27 % швидше), бо записує 7 817 стиснених бакетів.

3.2 Структура документів і типи полів

Структуру документів трьох схем показано на рисунку 3: плоский документ має `_id` і сім полів (`ts`, `unix`, OHLCV), time-series – ще поле `meta`, bucket-документ – масив `hours` (виведено перший з 24 елементів; Int64-поле `unix` `JSON.stringify` виводить як об'єкт).

```
$ mongosh --quiet --eval 'const P=(d)=>print(JSON.stringify(d)); P(db.candles_1m.findOne({
  }, {_id:0})); P(db.candles_1m_ts.findOne({}, {_id:0})); P(db.candles_1d_buckets.aggregat
  e([{$limit:1},{$project:{_id:0,date:1,n_hours:1,hours:{$slice:["$hours",1]}}}]).toArray(
  )[0])' bigdata_db
{"ts":"2012-01-01T00:01:00.000Z","unix":{"high":0,"low":1325376060,"unsigned":false},"open
↪ ":4.58,"high":4.58,"low":4.58,"close":4.58,"volume":0}
{"ts":"2012-01-01T00:01:00.000Z","meta":{"symbol":"BTCUSD"},"volume":0,"high":4.58,"unix":
↪ {"high":0,"low":1325376060,"unsigned":false},"low":4.58,"close":4.58,"open":4.58}
{"date":"2012-01-01T00:00:00.000Z","n_hours":24,"hours":[{"hour":0,"open":4.58,"high":4.58
↪ ,"low":4.58,"close":4.58,"volume":0}]}
```

Рисунок 3 – Перші документи колекцій candles_1m, candles_1m_ts і candles_1d_buckets (mongosh)

Типи полів визначено агрегацією `{$group: {_id: {$type: "$поле"}}}` по всіх документах кожної колекції (таблиця 2).

Таблиця 2 – BSON-типи полів

Колекція	Поля	BSON-тип
candles_1m, candles_1h	ts	date
candles_1m	unix	long
усі колекції свічок	open, high, low, close, volume	double
candles_1d, candles_1d_buckets	date / year, month, n_minutes, n_hours	date / int
candles_1d_buckets	hours	array вкладених документів
spark_daily_features	date / rv, hl_rel, ret_1d, vol_7d та ще 13 полів / n_minutes, n_flat, year, month, dow, is_complete	date / double / int
anomalies	ts, date / score, z_ret, z_vol, log_return та ще 6 полів / k, year, ts_epoch / rule, rule_version / in_stream_replay	date / double / int / string / bool

Кожне поле має єдиний тип, крім трьох полів Spark із null (п. 3.3). Час збережено як Date, а не рядок, тому працюють діапазонні фільтри, `$year`, `$month`, `$hour` і хронологічне сортування.

3.3 Пропущені значення

У 27 полях колекцій свічок немає жодного відсутнього, null чи NaN значення. У `spark_daily_features` NaN з Parquet замінено на null: 1 у `ret_1d` (перша доба без попередньої), по 2 у `vol_7d` і `vol_30d` (початок

ковзних вікон). Хвилини з нульовим обсягом (1 312 419, тобто 16,79 %) – не пропуски, а заповнення джерелом хвилин без угод, тому їх збережено.

4 Виконання запитів

Вісім запитів `find` охоплюють пошук, фільтрацію, сортування, обмеження і проєкцію (таблиця 3); час одного запуску виміряно в Python разом із передачею результату за наявності індексів розділу 6.

Таблиця 3 – Запити до бази bigdata_db

№	Призначення	Колекція	Оператори	Документів	Час, мс
Q1	доби 2021 року з close > 60 000	candles_1d	$gte, $lt, $gt, проєкція, sort	39	4,7
Q2	топ-10 діб за обсягом	candles_1d	sort(volume: −1), limit(10)	10	4,0
Q3	5 найнижчих low 12.03.2020, 12:00–13:00 UTC	candles_1m	діапазон ts, sort, limit(5)	5	1,1
Q4	5 хвилин з найбільшим обсягом	candles_1m	$gt, sort, limit(5)	5	2,1
Q5	повні доби з ret_1d < −0,10	spark_daily_features	рівність, $lt, sort, limit(10)	10	4,3
Q6	неповні доби (n_minutes < 1 440)	candles_1d	$lt, проєкція	2	2,4
Q7	кількість хвилин з volume = 0	candles_1m	countDocuments	1	370,9
Q8	листопад–грудень 2022 з flat_share = 0	spark_daily_features	$and, $in	2	10,5

Результат Q3 – одна година обвалу 12.03.2020 з 7,8 млн документів – наведено на рисунку 4.

```
$ mongosh --quiet --eval 'db.candles_1m.find({ts:{$gte:ISODate("2020-03-12T12:00:00Z"),$lt
  :ISODate("2020-03-12T13:00:00Z")}},{_id:0,ts:1,low:1,close:1,volume:1}).sort({low:1}).li
  mit(5).forEach(d => print(d.ts.toISOString(), String(d.low).padStart(8), String(d.close)
  .padStart(8), d.volume.toFixed(2).padStart(7)))' bigdata_db
2020-03-12T12:49:00.000Z     5850  5880.53   90.05
2020-03-12T12:52:00.000Z  5868.46  5868.46   76.24
2020-03-12T12:50:00.000Z  5874.02   5958.8   71.68
2020-03-12T12:54:00.000Z  5883.72  5924.93   66.28
2020-03-12T12:53:00.000Z  5887.69  5933.88  108.60
```

Рисунок 4 – Запит Q3: найнижчі ціни 12.03.2020 між 12:00 і 13:00 UTC

Мінімум години – 5 850 USD о 12:49 UTC при обсязі 90,0 BTC; вибірка за індексом `ts_1` займає 1,1 мс. Q5 дає найгірші повні доби – 11.04.2013 (лог-дохідність −0,664) і 12.03.2020 (−0,494); Q6 – дві неповні доби, 01.01.2012 (1 439 хвилин) і 11.11.2026 (218); Q8 – 09.11 і 10.11.2022, дні краху FTX. Найповільніший Q7 (370,9 мс) документів не читає: за `explain` це COUNT_SCAN по 1 312 419 ключах індексу `volume_-1`.

5 Агрегування

Склад і час шести конвеєрів – п'яти над свічками й ознаками Spark і конвеєра з `$lookup` над аномаліями (розділ 7) – наведено в таблиці 4.

Таблиця 4 – Конвеєри агрегації

Конвеєр	Колекція	Стадії	Рядків	Час, с
A1 місячні OHLC	candles_1m	$sort → $group ($first, $max, $min, $last, $sum) → $sort → $project	179	103,88
A2 обсяг за годиною доби, 2018+	candles_1m	$match → $group ($hour) → $sort → $project	24	65,93
A3 річна волатильність	spark_daily_features	$match → $group → $sort → $project	15	0,325
A4 розподіл діб за ціною	candles_1d	$bucket	7	0,290
A5 річний обсяг	candles_1d	$group → $sort	15	0,090
ANOM аномалії і ознаки Spark	anomalies	$group → $lookup → $unwind → $project → $sort	1 572	0,192

Для A1 час – медіана з 3 запусків, для решти – один запуск.

5.1 Місячні OHLC

A1 ресемплює хвилини в місячні свічки за правилом first/max/min/last/sum; `$sort` стоїть перед `$group`, бо інакше `$first` і `$last` брали б документи в порядку зберігання без гарантії хронології. Виконання в mongosh показано на рисунку 5.

```
$ mongosh --quiet --eval 'db.candles_1m.aggregate([{$sort:{ts:1}},{$group:{_id:{y:{$year:"
  $ts"},m:{$month:"$ts"}},open:{$first:"$open"},high:{$max:"$high"},low:{$min:"$low"},clos
  e:{$last:"$close"},volume:{$sum:"$volume"}}},{$sort:{_id:1}},{$match:{"_id.y":2021,"_id.
  m":{$lte:4}}}],{allowDiskUse:true}).forEach(d => print(d._id.y + "-" + String(d._id.m).p
  adStart(2,"0"), d.open, d.high, d.low, d.close, Math.round(d.volume)))' bigdata_db
2021-01 28999.63 42000 27734 33141.38 457461
2021-02 33110.32 58354.14 32333.01 45240.96 240351
2021-03 45261.56 61781.83 45020.25 58782.58 157899
2021-04 58789.96 64895.22 47004.2 57775.27 139624
```

Рисунок 5 – Конвеєр A1 у mongosh: місячні OHLC і обсяг, січень–квітень 2021 року

Порівняно з ресемплінгом pandas добових свічок для всіх 179 місяців різниця open, high, low і close дорівнює 0, а відносна різниця обсягу – $2{,}1 \cdot 10^{-16}$ (округлення суми з плаваючою комою).

5.2 Обсяг за годиною доби

A2 групує хвилини за `$hour` (UTC) починаючи з 2018 року, бо раніше велика частка хвилин без угод спотворювала середні. Результат подано на рисунку 6.

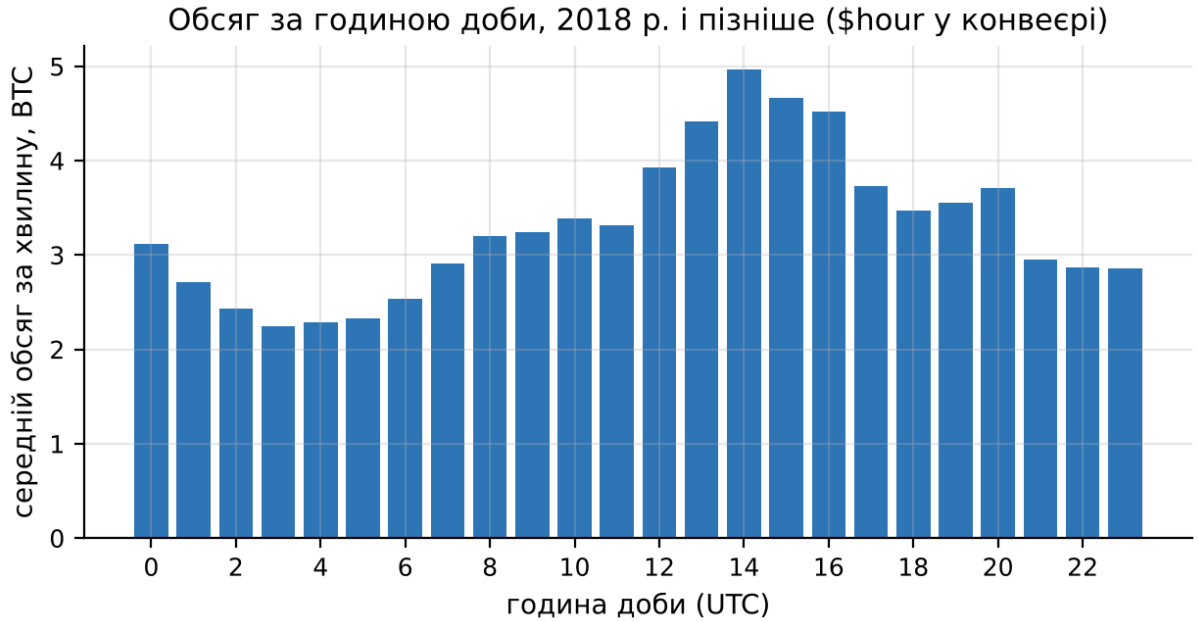


Рисунок 6 – Середній обсяг за хвилину за годиною доби (UTC), 2018–2026 роки

Найактивніша година – 14:00 UTC (4,96 BTC за хвилину), найспокійніша – 03:00 UTC (2,24 BTC), різниця в 2,2 раза. Пік 13:00–16:00

UTC збігається з відкриттям фондового ринку США: цілодобовий ринок біткоїна має виражену денну сезонність.

5.3 Річна волатильність і розподіл цін

A3 рахує річні показники за ознаками Spark для повних діб (`is_complete = 1`): середня `rv` знизилася з 0,1152 у 2013 році до 0,0200 у 2023 році і відтоді не перевищувала 0,0260. A4 (`$bucket`) розподіляє доби за ціною закриття: 1 332 доби в діапазоні 100–1 000 USD, 1 317 – у 10 000–50 000 USD і 217 діб понад 100 000 USD, перша з яких – 08.12.2024.

6 Індексація

Індекси створено для полів фільтрації і сортування запитів таблиці 5, а також `date_1` у `candles_1d` і `spark_daily_features` та `year_1` в `anomalies`. Фаза «до» – без вторинних індексів, «після» – з одним; після прогріву фіксуються медіани 5 запусків для `executionTimeMillis` з `explain("executionStats")` (сервер) і повного часу в Python. План «до» – COLLSCAN (з SORT, крім E5b), «після» – IXSCAN і FETCH (для E3 ще LIMIT, для ANOM1 ще SORT); кількість повернутих документів однакова, а переглянутих ключів дорівнює кількості документів, крім ANOM1 (338 ключів на 20 документів).

Зміну плану E1 у mongosh показано на рисунку 7: «до» отримано примусовим скануванням `hint({$natural: 1})` (один запуск, тому час відрізняється від медіани).

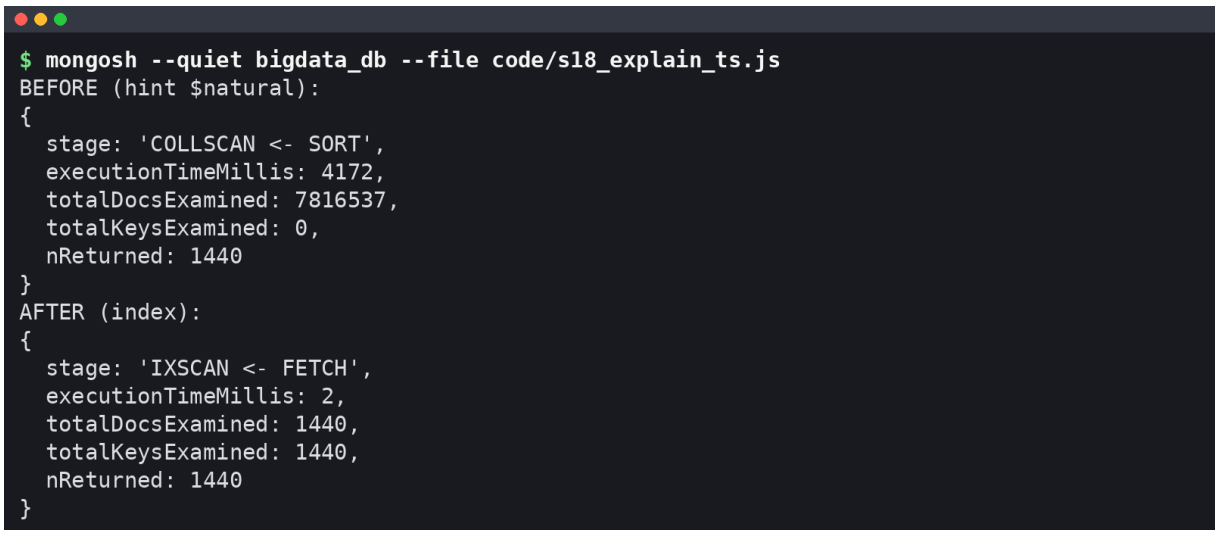


Рисунок 7 – План запиту E1 до (COLLSCAN) і після (IXSCAN) створення індексу ts_1

Результати всіх запитів наведено в таблиці 5 (E1–E3 – `candles_1m`, E4 – `candles_1d`, E5a – `candles_1h`, E5b – `candles_1d_buckets`, ANOM1 і ANOM2 – `anomalies`).

Таблиця 5 – Виконання запитів до і після створення індексу (медіана з 5 прогрітих запусків)

Запит	Індекс	explain, мс	Python, мс	Переглянуто док.
E1 доба 19.05.2021	ts_1	4 637 → 1	4 548,3 → 5,5	7 816 537 → 1 440
E2 травень 2021	ts_1	5 886 → 50	4 921,9 → 191,2	7 816 537 → 44 640
E3 топ-10 за volume	volume_-1	5 399 → 0	3 771,6 → 0,5	7 816 537 → 10
E4 2021, volume > 9 483,33	year_1_volume_-1	2 → 0	3,0 → 0,6	5 429 → 36
E5a 24 години	ts_1	86 → 0	73,5 → 0,6	130 276 → 24
E5b доба-bucket	date_1	2 → 0	3,9 → 0,5	5 429 → 1
ANOM1 2021, топ-20 за score	date_1_score_-1	3 → 0	3,6 → 1,0	5 437 → 20
ANOM2 доба 12.03.2020	date_1_score_-1	2 → 0	3,2 → 0,9	5 437 → 82

Час на сервері для всіх запитів подано на рисунку 8 у логарифмічній шкалі.



Рисунок 8 – Час виконання запитів на сервері (executionTimeMillis) без індексу і з індексом, медіана з 5 запусків

На хвилинній колекції `ts_1` скорочує вибірку доби з 4 637 до 1 мс на сервері, а за повним часом – з 4 548,3 до 5,5 мс (у 827 разів); топ-10 за обсягом прискорюється з 3 771,6 до 0,5 мс. Для травня 2021 року виграш менший (у 118 разів на сервері і в 26 разів у Python), бо час займають FETCH і передача 44 640 документів. Складений індекс `{year: 1, volume: -1}` відповідає правилу ESR, тому SORT зникає. В ANOM1 SORT лишився: діапазон дат охоплює 338 ключів, а порядок `score` гарантовано лише в межах однієї дати (для рівності на дату в ANOM2 сортування не потрібне). На малих колекціях виграш за повним часом – лише 2,3–3,4 мс.

Індекс `ts_1` займає 90,43 МіБ і будується 26,26 с, `volume_-1` – 107,41 МіБ і 31,29 с; разом з `_id` індекси `candles_1m` займають 280,07 МіБ при 383,05 МіБ даних. Вставка 200 000 хвилинних документів пакетами по 20 000 у копію колекції (медіана з 3 запусків) з `ts_1` і `volume_-1` іде зі швидкістю 66 824 док/с проти 120 458 без них (на 80,3 % повільніше), а побудова обох індексів після вставки – 1,183 с, тому історію вигідніше завантажувати без вторинних індексів. Без індексу bucket-схема (E5b)

повертає добу за 3,9 мс проти 73,5 мс у плоскої (E5a), з індексами обидві – за 0,5–0,6 мс.

6.1 Агрегація, яка не прискорюється індексом

A1 виконано по 3 рази з `ts_1` (медіана 103,88 с) і без нього з `hint({$natural: 1})` (104,08 с): різниця 0,2 % менша за розкид запусків (97,34–104,53 с). За `explain` обидва плани переглядають усі 7 816 537 документів; без індексу на диск виливається SORT (8 скидань), але в обох планах з диском працює і GROUP (3 908 268 скидань), що й визначає час. Місячні OHLC з `candles_1d` обчислюються за 0,108 с, тому місячні звіти доцільно рахувати з добових агрегатів.

7 Інтеграція з результатами Spark і ПЗ3

Колекцію `spark_daily_features` (5 429 документів, 24 поля) завантажено з добових ознак повторного прогону ПЗ2 (Spark), який відрізняється від попереднього лише в 137–178 рядках `rv`, `std_ret_1m` і `vol60_mean` (`rv` не більше ніж на 0,00065) і не змінив чисел звіту. Колекцію `anomalies` (5 437 документів, 0,16 с) завантажено з `pz3_anomalies.csv` ПЗ3. Правило v2 позначає хвилину аномальною, якщо модуль її лог-дохідності перевищує 9σ, де $\sigma = Q_{0,90}(|r|)/1{,}6449$ за активними хвилинами попередніх 7 діб, а обсяг вищий за тижневу медіану. Аномалії припадають на 1 572 доби; конвеєр `$lookup` з'єднує кожну з них з її ознаками Spark за полем `date` (рисунок 9).

```
$ mongosh --quiet --eval 'db.anomalies.aggregate([{$group:{_id:"$date",n_anom:{$sum:1},max
  _score:{$max:"$score"}}},{$lookup:{from:"spark_daily_features",localField:"_id",foreignF
  ield:"date",as:"f"}},{$unwind:"$f"},{$project:{_id:0,date:"$_id",n_anom:1,max_score:1,rv
  :"$f.rv",volume:"$f.volume"}},{$sort:{n_anom:-1}},{$limit:6}]).forEach(d => print(d.date
  .toISOString().slice(0,10), String(d.n_anom).padStart(4), d.max_score.toFixed(2).padStar
  t(6), d.rv.toFixed(4), Math.round(d.volume)))' bigdata_db
2020-03-12   82   5.85 0.2111 58513
2020-03-13   75   3.47 0.2784 54420
2019-04-02   66   4.60 0.0758 30686
2022-11-08   59   9.48 0.0933 12036
2018-11-14   58   4.86 0.0699 25122
2013-04-10   55  10.22 0.8632 20889
```

Рисунок 9 – Конвеєр $lookup: доби з найбільшою кількістю аномалій та їх ознаки Spark

Пару знайшли всі 1 572 доби. Перше місце посідає 12.03.2020 (82 аномалії, `rv` 0,211, лог-дохідність −0,494), друге – 13.03.2020 (75), четверте – 08.11.2022 (59, найбільший score 9,48 серед перших п'яти діб, крах FTX). За роками найбільше аномалій у 2019 (777) і 2023 (749) роках. Перекосу в бік тонкого ринку немає: у 2012–2016 роках позначено 0,058 % оцінених хвилин, у 2018+ – 0,096 %; виняток – 2013 рік (461 аномалія, 0,154 %, три дати з топ-10: 10.04, 02.10 і 19.11.2013). Доби з аномаліями порівняно з рештою повних діб у таблиці 6.

Таблиця 6 – Ознаки Spark для діб з аномаліями і решти повних діб

Група	Діб	Середнє rv	Медіана rv	Середнє hl_rel	Медіана обсягу, BTC
доби з аномаліями	1 572	0,0506	0,0353	0,0695	4 853,7
решта повних діб	3 855	0,0453	0,0356	0,0458	3 798,2

Медіана `rv` у групах майже однакова (0,0353 і 0,0356), а середнє `hl_rel` і медіанний обсяг у добах з аномаліями вищі в 1,5 і 1,3 раза: правило ПЗЗ знаходить раптові хвилинні стрибки з підвищеним обсягом, які розширюють денний діапазон, але не роблять увесь день волатильним.

8 Аналіз можливостей MongoDB для великих даних

Документна модель зберегла в одній базі дані чотирьох різних структур без міграцій, індекси переводять селективні запити з 4–6 с у мілісекунди, а Aggregation Framework достатній для ресемплінгу OHLC і

`$lookup`. Обмеження – повні проходи: агрегація по всіх хвилинах триває 66–104 с, індекс її не прискорює, а без шардування конвеєр не розподіляється між вузлами. У ПЗ2 на тій самій машині Spark local[*] будував добові OHLC з 7,8 млн хвилин за 4,547 с (медіана з 3 запусків разом з читанням CSV), pandas – за 4,884 с, Spark із закешованим DataFrame – за 0,618 с. Попри різні умови (добові групи, читання CSV, loadavg 8,6 проти 5–10) різниця з А1 (103,88 с) перевищує порядок. Тому MongoDB доцільна як шар зберігання і швидкого доступу: перерахунки історії виконувати в Spark, а в MongoDB завантажувати їх результати (добові агрегати, ознаки, аномалії) з індексами під типові запити.

Висновки

1. Встановлено MongoDB Community Server 8.3.7, mongosh 2.14.1 і PyMongo 4.19.0; Compass у контейнері без дисплея недоступний, його операції виконано в mongosh.
2. Створено базу `bigdata_db` із семи колекцій; time-series колекція займає 69,31 МіБ проти 383,05 МіБ для плоскої, але за часом запитів схеми рівноцінні, і основною обрано плоску.
3. Імпортовано 7 816 537 хвилинних документів за 117,7 с (78,5 тис. док/с); пропусків у 27 полях свічок немає, в ознаках Spark 5 значень null, нульовий обсяг 16,79 % хвилин є заповненням сітки.
4. Вісім запитів `find`, крім Q7 (COUNT_SCAN, 370,9 мс), виконуються за 1,1–10,5 мс.
5. Шість конвеєрів агрегації дали місячні OHLC, що збігаються з pandas для 179 місяців, денну сезонність обсягу (пік о 14:00 UTC) і зниження середньої `rv` з 0,1152 до 0,0200.
6. Індекс `ts_1` скоротив вибірку доби з 4 637 до 1 мс на сервері, але разом з `volume_-1` уповільнив запис на 80,3 %; повна агрегація А1 від індексу не виграє (103,88 проти 104,08 с).

7. Імпортовано результати П32 (5 429 діб ознак Spark) і П33 (5 437 аномалій, k = 9) і з'єднано їх через `$lookup`.
8. MongoDB ефективна як сховище результатів з мілісекундним доступом за індексами, а повні проходи – у Spark (добові OHLC за 4,547 с проти 103,88 с для місячних OHLC у MongoDB).